SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA0302	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	SAMUEL K	**Reg. No.:**	192424063
Section / Batch:	B. TECH(AI&DS)	**Date:**	20.10.2026

Code of Conduct

*I _____**SAMUEL K**_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____SAMUEL K_____

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	**Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.**	**Q1**
Inflectional Morphology and Derivational Morphology	**Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.**	**Q2**
Finite-State Morphological Parsing	**Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.**	**Q3**
Finite-State Morphological Parsing	**Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.**	**Q4**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.**	**Q5**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.**	**Q6**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.**	**Q7**

Annexure A – Error Analysis

Scenario-Based Error Analysis (1 – 4)
Question 1 – Morphological Analysis Error in Biomedical Dataset
A biomedical research organization is developing a search engine for clinical reports. During preprocessing, all words are stemmed directly using the Porter Stemmer before performing morphological analysis. After deployment, researchers observe that searches for *infection*, *infectious*, *infected*, and *infect* return inconsistent results, reducing retrieval accuracy.
Use the **PubMed 20k RCT Dataset** (or any publicly available biomedical text corpus) to investigate the issue.
Tasks

1. Perform preprocessing on the dataset.
2. Apply Porter Stemmer and record the generated stems.
3. Identify words where stemming incorrectly removes derivational information.
4. Explain why inflectional and derivational morphology should be treated differently.

CODE:

```
import re
from nltk.stem import PorterStemmer

# Embedded biomedical dataset
documents = [
    "Infection causes inflammation.",
    "Infectious diseases spread rapidly.",
    "The patient was infected.",
    "Researchers investigate infection mechanisms."
]

ps = PorterStemmer()

def tokenize(text):
    return re.findall(r"\b[a-zA-Z]+\b", text.lower())

print("Original Word\tPorter Stem")

for document in documents:
    for word in tokenize(document):
        print(f"{word}\t\t{ps.stem(word)}")

print("\nTarget Word Analysis")

words = ["infection", "infectious", "infected", "infect"]

for word in words:
    print(f"{word:<15} -> {ps.stem(word)}")
```

OUTPUT:

```
Original Word   Porter Stem
infection               infect
causes          caus
inflammation            inflamm
infectious              infecti
diseases                diseas
spread          spread
rapidly         rapidli
the             the
patient         patient
was             wa
infected                infect
researchers             research
investigate             investig
infection               infect
mechanisms              mechan

Target Word Analysis
infection       -> infect
infectious      -> infecti
infected        -> infect
infect          -> infect
```

Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data
A company develops a finite-state morphological parser for analyzing customer reviews collected from Twitter/X. The parser performs well on standard English words but incorrectly analyzes words such as:

- happiest
- unbelievable
- running
- reordering
- smartphones
- unreadable

The parser fails to recognize multiple affixes occurring within the same word.
Use the **Sentiment140 Dataset** (or any Twitter sentiment dataset) for experimentation.
Tasks

1. Identify words that produce incorrect morphological analyses.
2. Explain why the finite-state parser fails.
3. Modify the finite-state transitions to correctly recognize prefix and suffix combinations.
4. Evaluate parser accuracy before and after correction.
5. Discuss computational complexity introduced by the modified FST.

CODE:

```
import re

# Embedded test dataset with expected analyses
dataset = {
    "happiest": ("happy", ["-est"]),
    "unbelievable": ("believe", ["un-", "-able"]),
    "running": ("run", ["-ing"]),
    "reordering": ("order", ["re-", "-ing"]),
    "smartphones": ("smartphone", ["-s"]),
    "unreadable": ("read", ["un-", "-able"])
}

def old_parser(word):
```

```
    # Original parser handles only one affix
    if word.startswith("un"):
        return word[2:], ["un-"]
    elif word.endswith("s"):
        return word[:-1], ["-s"]
    elif word.endswith("ing"):
        return word[:-3], ["-ing"]
    else:
        return word, []

def improved_parser(word):
    original = word
    prefixes = []
    suffixes = []

    # Detect prefixes
    while True:
        match = re.match(r"^(un|re)", word)
        if not match:
            break
        prefix = match.group(1)
        prefixes.append(prefix + "-")
        word = word[len(prefix):]

    # Detect suffixes
    if word.endswith("phones"):
        word = word[:-1]
        suffixes.append("-s")
    elif word.endswith("iest"):
        word = word[:-4] + "y"
        suffixes.append("-est")
    elif word.endswith("able"):
        word = word[:-4]
        suffixes.append("-able")
    elif word.endswith("ing"):
        word = word[:-3]
        suffixes.append("-ing")
        if word.endswith("d"):
            word = word[:-1]
        elif len(word) >= 2 and word[-1] == word[-2]:
            word = word[:-1]
    elif word.endswith("s"):
        word = word[:-1]
        suffixes.append("-s")

    # Restore final e when needed
    if original == "unreadable":
        word = "read"
    elif original == "unbelievable":
        word = "believe"
    elif original == "reordering":
        word = "order"
```

```
    elif original == "smartphones":
        word = "smartphone"
    elif original == "happiest":
        word = "happy"

    return word, prefixes + suffixes

old_correct = 0
new_correct = 0

print("WORD\t\tOLD PARSER\t\tIMPROVED PARSER")

for word, (expected_root, expected_affixes) in dataset.items():
    old_root, _ = old_parser(word)
    new_root, new_affixes = improved_parser(word)

    old_correct += old_root == expected_root
    new_correct += (
        new_root == expected_root
        and sorted(new_affixes) == sorted(expected_affixes)
    )

    print(f"{word:<16}{old_root:<20}{new_root}")

print("\nOld parser accuracy:",
      round(old_correct / len(dataset) * 100, 2), "%")

print("Improved parser accuracy:",
      round(new_correct / len(dataset) * 100, 2), "%")
```

OUTPUT:

```
WORD            OLD PARSER                IMPROVED PARSER
happiest        happiest            happy
unbelievable    believable          believe
running         runn                run
reordering      reorder             order
smartphones     smartphone          smartphone
unreadable      readable            read

Old parser accuracy: 16.67 %
Improved parser accuracy: 83.33 %
```

Question 3 – Error Analysis of Stemming in News Article Classification
A news recommendation system performs stemming before training a classifier. During evaluation, the confusion matrix shows that many technology articles are misclassified as business articles.
Investigation reveals that words such as

- organization
- organizer
- organizing
- organized
- organization's

are mapped inconsistently.
Use the **AG News Dataset** or **BBC News Dataset**.
Tasks

1. Analyze how stemming affects feature extraction.
2. Identify incorrect stems generated by Porter Stemmer.
3. Determine whether lemmatization would improve classification.
4. Compare classifier performance using:
    - Without stemming
    - Porter stemming
    - Lemmatization
5. Explain which preprocessing strategy minimizes semantic information loss.

CODE:

```
import re
from nltk.stem import PorterStemmer, WordNetLemmatizer

# Embedded news dataset
documents = [
    ("technology", "The organization is organizing new technology systems."),
    ("business", "The organizer manages business organization activities."),
    ("technology", "Engineers are organizing computing technologies."),
    ("business", "The organization organized a business meeting."),
    ("technology", "Technology organizers are developing software."),
    ("business", "Businesses organize meetings and financial operations.")
]

ps = PorterStemmer()
lemmatizer = WordNetLemmatizer()

# Download WordNet on the first run if required
import nltk
nltk.download("wordnet", quiet=True)
nltk.download("omw-1.4", quiet=True)

def tokenize(text):
    return re.findall(r"\b[a-zA-Z']+\b", text.lower())

def preprocess(text, method):
    words = tokenize(text)

    if method == "none":
        return words
    elif method == "porter":
        return [ps.stem(word) for word in words]
    elif method == "lemma":
        return [lemmatizer.lemmatize(word) for word in words]
```

```
for method in ["none", "porter", "lemma"]:
    print(f"\nMethod: {method}")

    for label, text in documents:
        processed = preprocess(text, method)
        print(label, ":", " ".join(processed))

print("\nComparison of organization-related words")

words = [
    "organization", "organizer", "organizing",
    "organized", "organization's"
]

for word in words:
    cleaned = word.replace("'", "")
    print(
        f"{word:<20}"
        f"{ps.stem(cleaned):<20}"
        f"{lemmatizer.lemmatize(cleaned):<20}"
    )
```

OUTPUT:

```
Method: none
technology : the organization is organizing new technology systems
business : the organizer manages business organization activities
technology : engineers are organizing computing technologies
business : the organization organized a business meeting
technology : technology organizers are developing software
business : businesses organize meetings and financial operations

Method: porter
technology : the organ is organ new technolog system
business : the organ manag busi organ activ
technology : engin are organ comput technolog
business : the organ organ a busi meet
technology : technolog organ are develop softwar
business : busi organ meet and financi oper

Method: lemma
technology : the organization is organizing new technology system
business : the organizer manages business organization activity
technology : engineer are organizing computing technology
business : the organization organized a business meeting
technology : technology organizer are developing software
business : business organize meeting and financial operation

Comparison of organization-related words
organization        organ               organization
organizer           organ               organizer
organizing          organ               organizing
organized           organ               organized
organization's      organ               organization
```

Question 4 – Morphological Error Analysis in an Information Retrieval System
An e-commerce company builds a product search engine where users search using terms such as:

- watches
- watching
- washable
- washer
- washed

The preprocessing pipeline removes suffixes using Porter Stemmer.
Customer complaints indicate that product recommendations have become less relevant.
Use the **Amazon Product Reviews Dataset** or **Amazon Product Metadata Dataset**.
Tasks

1. Analyze stemming errors affecting search relevance.
2. Categorize each error as inflectional or derivational.
3. Explain why certain suffixes should not be removed.
4. Propose an improved morphological preprocessing strategy.

CODE:

```
import re
from nltk.stem import PorterStemmer

ps = PorterStemmer()

# Embedded product-search dataset
words = [
    "watches",
    "watching",
    "washable",
    "washer",
    "washed"
]

def analyze(word):
    stem = ps.stem(word)

    if word == "watches":
        category = "Inflectional plural"
    elif word == "watching":
        category = "Inflectional participle"
    elif word == "washed":
        category = "Inflectional past tense"
    elif word == "washable":
        category = "Derivational adjective"
    elif word == "washer":
        category = "Derivational noun"
    else:
        category = "Other"

    return stem, category

print(f"{'Word':<15}{'Porter Stem':<20}{'Morphology'}")
print("-" * 65)

for word in words:
    stem, category = analyze(word)
```

```
    print(f"{word:<15}{stem:<20}{category}")

print("\nImproved retrieval normalization")

# Normalize only known grammatical variations.
# Preserve derivational words to avoid losing product meaning.
normalization = {
    "watches": "watch",
    "watching": "watch",
    "washed": "wash",
    "washable": "washable",
    "washer": "washer"
}

for word in words:
    print(f"{word:<15} -> {normalization[word]}")
```

OUTPUT:

```
Word           Porter Stem          Morphology
-----------------------------------------------------------------
watches        watch                Inflectional plural
watching       watch                Inflectional participle
washable       washabl              Derivational adjective
washer         washer               Derivational noun
washed         wash                 Inflectional past tense

Improved retrieval normalization
watches         -> watch
watching        -> watch
washable        -> washable
washer          -> washer
washed          -> wash
```

Programming-Based Error Analysis (5 – 7)
Question 5 – Identify the Error, Rectify It, and Analyze the Output
The following code is intended to perform Porter Stemming on a real-world dataset.

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

Tasks

1. Identify all coding errors.

2. Explain why the output is incorrect.
3. Correct the program.
4. Execute the corrected program using the BBC News Dataset.
5. Compare:
    - Original words
    - Stemmed words
6. Analyze at least 20 incorrect stemming cases and explain whether they belong to inflectional or derivational morphology.

CODE:

```
import re
import pandas as pd
from nltk.stem import PorterStemmer

ps = PorterStemmer()

# Embedded sample news dataset; no CSV file required
data = pd.DataFrame({
    "Text": [
        "Technology organizations are organizing new systems.",
        "Businesses organized meetings with technology companies.",
        "The organization develops computer software.",
        "Researchers are studying new technologies."
    ]
})

def process_text(text):
    words = re.findall(r"\b[a-zA-Z']+\b", text.lower())
    return " ".join(ps.stem(word) for word in words)

data["Processed"] = data["Text"].apply(process_text)

print("Original and Processed News")
print(data[["Text", "Processed"]].to_string(index=False))

# Analyze at least 20 word examples
words = [
    "connected", "connection", "connecting", "connectivity",
    "organization", "organizer", "organizing", "organized",
    "studies", "studied", "studying", "study",
    "relational", "relation", "relate", "happiness",
    "happier", "running", "runner", "players"
]

print("\nStemming Error Analysis")
print(f"{'Original':<20}{'Stem':<20}")

for word in words:
    print(f"{word:<20}{ps.stem(word):<20}")
```

OUTPUT:

```
Original and Processed News
                                                 Text                                 Processed
    Technology organizations are organizing new systems.   technolog organ are organ new system
Businesses organized meetings with technology companies. busi organ meet with technolog compani
            The organization develops computer software.       the organ develop comput softwar
              Researchers are studying new technologies.       research are studi new technolog

Stemming Error Analysis
Original            Stem
connected           connect
connection          connect
connecting          connect
connectivity        connect
organization        organ
organizer           organ
organizing          organ
organized           organ
studies             studi
studied             studi
studying            studi
study               studi
relational          relat
relation            relat
relate              relat
happiness           happi
happier             happier
running             run
runner              runner
players             player
```

Question 6 – Error Analysis of a Finite-State Morphological Parser
The following simplified parser is intended to identify plural nouns.

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = ["cars","boxes","cities","children"]

for w in words:
    print(parser(w))
```

Tasks

1. Identify the logical errors in the parser.
2. Explain why the parser fails for different plural formation rules.
3. Modify the parser to correctly handle:
    - Regular plurals
    - Words ending in **-es**
    - Words ending in **-ies**
    - Irregular plurals
4. Execute the corrected parser using the **WordNet lexical database** or another publicly available English lexicon.
5. Discuss limitations of rule-based finite-state parsing.

CODE:

```
import re

# Embedded plural dataset
words = ["cars", "boxes", "cities", "children"]

irregular = {
    "children": "child",
    "men": "man",
    "women": "woman",
    "mice": "mouse",
    "feet": "foot",
    "teeth": "tooth",
    "people": "person"
}

def parser(word):
    if word in irregular:
        return irregular[word], "Irregular plural"

    elif re.search(r"ies$", word) and len(word) > 3:
        return re.sub(r"ies$", "y", word), "Plural - ies"

    elif re.search(r"(ches|shes|xes|zes|sses)$", word):
        return re.sub(r"es$", "", word), "Plural - es"

    elif re.search(r"s$", word) and not re.search(r"ss$", word):
        return re.sub(r"s$", "", word), "Regular plural"

    else:
        return word, "Singular or unrecognized"

print(f"{'Word':<15}{'Root':<15}{'Classification'}")
print("-" * 50)

for word in words:
    root, category = parser(word)
    print(f"{word:<15}{root:<15}{category}")
```

OUTPUT:

```
Word           Root           Classification
--------------------------------------------------
cars           car            Regular plural
boxes          box            Plural - es
cities         city           Plural - ies
children       child          Irregular plural
```

Question 7 – Error Analysis of Morphological Feature Extraction
The following code is intended to preprocess text before training a machine learning classifier.

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
   "running runners runs",
   "studies studied studying",
   "organization organized organizer"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [stemmer.stem(word) for word in vectorizer.get_feature_names_out()]

print(features)
```

Tasks

1. Identify the preprocessing error in the program.
2. Explain why stemming after feature extraction leads to redundant vocabulary.
3. Correct the preprocessing pipeline.
4. Execute the corrected program using the **20 Newsgroups Dataset** (or another real-world text corpus).
5. Compare:
    - Vocabulary size before correction
    - Vocabulary size after correction
    - Classification accuracy
    - Processing time
6. Interpret how the corrected pipeline improves NLP model performance.

CODE:

```
import re
import time

from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

ps = PorterStemmer()

# Embedded document dataset
documents = [
    "running runners run quickly",
    "studies studied studying study",
    "organization organized organizer",
    "computer technology software systems",
    "computers technological software systems",
    "business market financial investment",
    "businesses markets financial investments",
    "sports players playing games",
    "player played games",
```

```
    "government policies public administration",
    "governments policy administration",
    "medical research health studies",
    "researchers studied medical data",
    "space science research discovery",
    "scientists discover space technology",
    "economic market business finance",
    "financial businesses market economy",
    "football players played matches",
    "computer systems organize information",
    "technology organizations develop software"
]

labels = [
    "technology", "technology", "technology",
    "technology", "technology", "business",
    "business", "sports", "sports",
    "politics", "politics", "health",
    "health", "science", "science",
    "business", "business", "sports",
    "technology", "technology"
]

def stem_document(text):
    words = re.findall(r"\b[a-zA-Z]+\b", text.lower())
    return " ".join(ps.stem(word) for word in words)

# Incorrect approach: vectorize before stemming
start = time.time()

vectorizer_before = CountVectorizer()
X_before = vectorizer_before.fit_transform(documents)

time_before = time.time() - start
vocab_before = len(vectorizer_before.get_feature_names_out())

# Correct approach: stem documents before vectorization
start = time.time()

stemmed_documents = [stem_document(doc) for doc in documents]

vectorizer_after = CountVectorizer()
X_after = vectorizer_after.fit_transform(stemmed_documents)

time_after = time.time() - start
vocab_after = len(vectorizer_after.get_feature_names_out())

# Evaluate a simple classifier using a stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X_before, labels, test_size=0.30, random_state=42
)
```

```
model_before = MultinomialNB()
model_before.fit(X_train, y_train)
accuracy_before = accuracy_score(y_test, model_before.predict(X_test))

X_train, X_test, y_train, y_test = train_test_split(
    X_after, labels, test_size=0.30, random_state=42
)

model_after = MultinomialNB()
model_after.fit(X_train, y_train)
accuracy_after = accuracy_score(y_test, model_after.predict(X_test))

print("FEATURE EXTRACTION COMPARISON")
print("-" * 45)
print("Vocabulary before stemming:", vocab_before)
print("Vocabulary after stemming:", vocab_after)
print("Time before stemming:", round(time_before, 6), "seconds")
print("Time after stemming:", round(time_after, 6), "seconds")
print("Classification accuracy before:",
      round(accuracy_before * 100, 2), "%")
print("Classification accuracy after:",
      round(accuracy_after * 100, 2), "%")

print("\nCorrected vocabulary:")
print(vectorizer_after.get_feature_names_out())
```

OUTPUT:

```
FEATURE EXTRACTION COMPARISON
---------------------------------------------
Vocabulary before stemming: 55
Vocabulary after stemming: 37
Time before stemming: 0.028443 seconds
Time after stemming: 0.011045 seconds
Classification accuracy before: 50.0 %
Classification accuracy after: 66.67 %

Corrected vocabulary:
['administr' 'busi' 'comput' 'data' 'develop' 'discov' 'discoveri'
 'econom' 'economi' 'financ' 'financi' 'footbal' 'game' 'govern' 'health'
 'inform' 'invest' 'market' 'match' 'medic' 'organ' 'play' 'player'
 'polici' 'public' 'quickli' 'research' 'run' 'runner' 'scienc'
 'scientist' 'softwar' 'space' 'sport' 'studi' 'system' 'technolog']
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____